Ejercicio 1: Sistema de Sensores IoT de un Dispositivo Móvil

En las aplicaciones móviles actuales es imprescindible interactuar con los sensores hardware del dispositivo.

1. Crea una superclase open class Sensor(val id: String, open val frecuenciaHz: Int) con la propiedad var activo: Boolean = false.
2. Sobrescribe en Sensor los métodos de Any:
    - toString(): Devuelve "[Sensor ID: $id] Frec: ${frecuenciaHz}Hz | Activo: $activo".
    - equals(other: Any?): Dos sensores son iguales si tienen el mismo id.
    - hashCode(): Basado en el id.
3. Crea la subclase SensorAcelerometro que herede de Sensor.
    - Modifica frecuenciaHz para que sea una propiedad override var que pueda cambiarse en tiempo de ejecución.
    - Añade un constructor secundario que reciba solo el id y asigne una frecuencia por defecto de 50 Hz.
    - Añade el método leerDatos(): String que retorne una cadena con valores simulados solo si está activo.

Ejercicio 2: Gestor de Notificaciones Push y Alertas Locales

En Android, el canal de comunicación con el usuario se realiza mediante notificaciones.

1. Crea la interfaz Notificable con:
    - Una propiedad val prioridad: String (abstracta).
    - Un método abstracto enviar(mensaje: String).
    - Un método con implementación por defecto mostrarLog(mensaje: String) que imprima "[LOG ${prioridad}]: $mensaje".
2. Crea la superclase open class NotificacionBase(val idNotificacion: Int, val titulo: String).
3. Implementa dos subclases que hereden de NotificacionBase e implementen Notificable:
    - NotificacionPush: Contiene la propiedad val tokenDispositivo: String.

- NotificacionLocal: Permite instanciarse sin idNotificacion mediante un constructor secundario que genere un ID aleatorio o por defecto.

4. Sobrescribe en ambas el método enviar(mensaje: String) haciendo uso previo de super.mostrarLog(...).

Ejercicio 3: Jerarquía de Vistas UI Custom para Android (View System)

Simula una jerarquía básica del sistema de vistas de Android (View, TextView, Button).

1. open class ComponenteView(val id: String):
    - Propiedades: open val esClickable: Boolean = false, var ancho: Int = 0, var alto: Int = 0.
    - Métodos: open fun dibujar() (imprime ID y dimensiones).
2. open class TextView(id: String, open var texto: String) : heredando de ComponenteView.
    - Sobrescribe dibujar() invocando super.dibujar() y añadiendo el texto que renderiza.
3. class Button : heredando de TextView.
    - Redefine esClickable a true mediante override val.
    - Posee dos constructores secundarios:
        - Uno que reciba id y texto.
        - Otro que reciba solo id y ponga el texto por defecto a "Aceptar".
    - Sobrescribe dibujar() añadiendo un marco alrededor del texto.

Ejercicio 4: Motor de Persistencia de Datos (Room / SharedPreferences)

Modela un sistema de persistencia donde existan fuentes de datos locales y remotas.

1. Interfaz CargadorDatos:
    - Método fun cargar(): String
    - Método fun guardar(datos: String): Boolean
2. open class FuenteDatosBase(val nombreFuente: String):

- Propiedad open val esRemota: Boolean = false.
- Sobrescribe toString() para mostrar el nombre de la fuente y si es remota.

3. Subclase AlmacenamientoPreferencias(nombreFuente: String) : hereda de FuenteDatosBase e implementa CargadorDatos.
    - Maneja internamente una propiedad privada var cache: String = "".
4. Subclase ServicioApiRest : hereda de FuenteDatosBase e implementa CargadorDatos.
    - No usa constructor primario en la firma de la clase, sino un constructor secundario que llama a super("API_REST_CLOUD") y establece esRemota en true.

Ejercicio 5: Pasarela de Pagos In-App (Google Pay vs Tarjeta)

Implementa la lógica de pagos dentro de una app con procesamiento de transacciones.

1. Interfaz ProcesadorPago:
    - fun procesar(monto: Double): Boolean
    - fun emitirRecibo(monto: Double) (con implementación por defecto que imprima el ticket con el monto y la fecha simulada).
2. open class Transaccion(open val idTransaccion: String, open val moneda: String = "EUR")
    - Sobrescribe equals() y hashCode() basándose en idTransaccion.
3. Subclase PagoGooglePay:
    - Hereda de Transaccion e implementa ProcesadorPago.
    - Propiedad extra val correoCuenta: String.
    - Sobrescribe moneda mediante override val permitiendo cambiarla en el constructor primario.
4. Subclase PagoTarjeta:
    - Hereda de Transaccion e implementa ProcesadorPago.
    - Posee constructor primario (idTransaccion: String, val numeroTarjeta: String, val cvv: Int) y un constructor secundario que no recibe idTransaccion y genera uno automáticamente concatenando "TX-" con los últimos 4 dígitos de la tarjeta.

Ejercicio 6: Reproductor Multimedia (MediaPlayer / Audio vs Video)

Diseña las clases para un reproductor multimedia de Android.

1. Interfaz ControlReproduccion:
    - fun reproducir()
    - fun pausar()
    - fun detener() { println("Reproducción detenida por completo.") } (Implementación por defecto).
2. open class ArchivoMultimedia(val uri: String, open val duracionSegundos: Int):
    - Sobrescribe toString() indicando la URI y la duración formateada en minutos/segundos.
3. Subclase PistaAudio(uri: String, duracion: Int, val artista: String) : hereda de ArchivoMultimedia e implementa ControlReproduccion.
    - Permite constructor secundario que reciba solo uri y asigne duracion = 0 y artista = "Desconocido".
4. Subclase VideoStreaming(uri: String, duracion: Int, val calidad: String) : hereda de ArchivoMultimedia e implementa ControlReproduccion.
    - Sobrescribe duracionSegundos con override var para poder actualizar la duración a medida que el vídeo se descarga por streaming.

Ejercicio 7: Modelado de Autenticación de Usuarios en App

Modela el sistema de cuentas de usuario para una app con login básico y biológico.

1. open class UsuarioBase(val idUsuario: String, val email: String):
    - Propiedad open val nivelAcceso: Int = 1.
    - Sobrescribe equals y hashCode usando únicamente el email.
    - Método open fun validarPermiso(nivelRequerido: Int): Boolean que compare nivelAcceso >= nivelRequerido.
2. Subclase UsuarioBiometrico:
    - Hereda de UsuarioBase.

- Promueve nivelAcceso a 2 sobrescribiendo la propiedad.
- Posee una propiedad val huellaRegistrada: Boolean.
- Posee constructor secundario que recibe idUsuario y email, asignando huellaRegistrada = true por defecto.

3. Subclase UsuarioAdministrador:
    - Hereda de UsuarioBase.
    - Promueve nivelAcceso a 10.
    - Sobrescribe validarPermiso para que **siempre** retorne true, independientemente del nivel requerido.

Ejercicio 8: Motor de Renderizado de Mapas (Marcadores Custom)

En aplicaciones de mapas (como Google Maps API), colocamos distintos marcadores según el tipo de punto de interés.

1. Interfaz DibujableEnMapa:
    - val latitud: Double
    - val longitud: Double
    - fun renderizarIcono()
    - fun obtenerCoordenadas(): String { return "($latitud,$longitud)" }
2. open class MarcadorBase(override val latitud: Double, override val longitud: Double, val titulo: String) : implementa DibujableEnMapa.
    - Sobrescribe toString() con "[Marcador] $titulo en ${obtenerCoordenadas()}".
3. Subclase MarcadorGasolineras : hereda de MarcadorBase.
    - Propiedad val precio95: Double.
    - Sobrescribe renderizarIcono() imprimiendo un icono de surtidor y el precio.
4. Subclase MarcadorRestaurante : hereda de MarcadorBase.
    - Propiedad val tenedores: Int.
    - Posee constructor secundario que asigna latitud y longitud a 0.0 (por si la dirección aún no se ha geolocalizado).

Ejercicio 9: Sistema de Tareas en Segundo Plano (WorkManager Simulation)

Simula el comportamiento de ejecución de tareas en background de la librería WorkManager de Android.

1. Interfaz Ejecutable:
    - fun ejecutar(): Boolean
    - fun cancelar() { println("Tarea cancelada.") }
2. open class TareaWorkManager(val idWorker: String):
    - Propiedad open val requiereRed: Boolean = false.
    - Propiedad var reintentos: Int = 0.
    - Sobrescribe toString() mostrando el ID y los reintentos.
3. Subclase TareaSincronizacionServidor : hereda de TareaWorkManager e implementa Ejecutable.
    - Redefine requiereRed a true (override val).
    - Sobrescribe ejecutar(): si requiereRed es true, simula la subida de datos y retorna true.
4. Subclase TareaLimpiezaCache : hereda de TareaWorkManager e implementa Ejecutable.
    - Posee constructor secundario que genera un idWorker automático tipo "CLEAN_TASK_" + Random.nextInt().

Ejercicio 10: Gestión de Dispositivos Bluetooth BLE (Dispositivos Conectables)

Crea una jerarquía de gestión de dispositivos Bluetooth Low Energy (comunes en apps de salud/deporte).

1. open class DispositivoBLE(val macAddress: String, open val nombre: String):
    - Propiedad open val nivelBateria: Int = 100.
    - Sobrescribe los métodos de Any:

        - equals() y hashCode() basados en macAddress.
        - toString() devolviendo "$nombre ($macAddress) - Batería: $nivelBateria %".

2. Interfaz Conectable:
    - fun conectar(): Boolean
    - fun desconectar()
    - fun verificarEstado() { println("Estado de la conexión: Estable.") }
3. Subclase BandaCardiaca:
    - Hereda de DispositivoBLE e implementa Conectable.
    - Modifica nivelBateria mediante override var para poder actualizarlo en tiempo real.
    - Posee constructor primario completo y constructor secundario que recibe macAddress y pone por defecto el nombre a "Banda HRM Generica".
4. Subclase RelojInteligente:
    - Hereda de DispositivoBLE e implementa Conectable.
    - Posee la propiedad val pantallaOled: Boolean.
    - Sobrescribe conectar() haciendo primero la llamada por defecto a verificarEstado() de la interfaz.